UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 5 – Function, Array, Object di JavaScript

Nama Mahasiswa	:	Nanik Wijayanti
NIM	:	23051430006
Kelas	:	A2
Tanggal Praktikum	:	13 Maret 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 5 (Pertemuan 5)
Topik Utama	Function, Array, Object di JavaScript
Sub-Topik	Modularitas kode (Function), pengelolaan data list (Array), dan struktur data kompleks (Object).
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Maret 2026
Nama Mahasiswa	Nanik Wijayanti
NIM	23051430006
Kelas	A2

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep Function untuk menghindari pengulangan kode (DRY - Don't Repeat Yourself).
2	Menguasai penggunaan Array untuk menyimpan sekumpulan data (misal: daftar cacat produksi).
3	Memahami Object untuk merepresentasikan entitas nyata (misal: Spesifikasi Mesin) dalam bentuk data.
4	Mampu melakukan iterasi (perulangan) terhadap Array dan Object.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Console Browser	Browser Modern	Untuk melakukan pengujian logika JavaScript, debugging,

			serta menampilkan hasil eksekusi kode secara langsung selama praktikum
5	Live Server	Ekstensi VS Code	Ekstensi untuk menjalankan server lokal agar perubahan kode web dapat terlihat secara *real-time* di browser

D. DASAR TEORI

Tuliskan ringkasan konsep/teori yang menjadi landasan pada modul ini. Ditulis dengan bahasa sendiri, bukan salinan mentah dari modul. Minimal 3 paragraf.

Di dalam dunia pemrograman, khususnya JavaScript, kita seringkali dihadapkan pada situasi di mana kode yang kita tulis menjadi berantakan karena terlalu banyak pengulangan. Di sinilah peran Function menjadi penting. Function seperti sebuah *Standard Operating Procedure* (SOP) di pabrik, kita hanya perlu merumuskan langkah-langkah kerja satu kali secara mendetail, lalu kapan pun operator butuh menjalankan prosedur tersebut, mereka cukup memanggil "nama SOP" nya saja. Hal ini kita memegang prinsip efisiensi agar tidak membuang waktu menulis logika yang sama berulang kali.

Selanjutnya, kita perlu wadah untuk menampung data yang jumlahnya banyak, seperti daftar defect atau list inventaris barang. Array hadir sebagai solusi "lemari arsip" yang sangat rapi. Dengan array, bisa menyimpan berbagai macam data dalam satu variabel saja, di mana setiap data memiliki nomor urut atau indeks. Hal ini memudahkan kita saat ingin melakukan pengecekan masal, misalnya menghitung total stok atau mencari item tertentu dari ribuan data yang tersimpan tanpa harus membuat variabel baru satu per satu.

Namun, data di dunia nyata seringkali lebih kompleks daripada sekadar daftar angka atau kata. Sebuah mesin, misalnya, tidak hanya punya satu nilai, tapi punya merk, tahun produksi, kapasitas daya, hingga status pemeliharaan. Object memungkinkan kita membungkus semua informasi detail tersebut ke dalam satu identitas yang utuh. Jika array adalah daftar baris di Excel, maka object adalah satu baris lengkap yang memiliki berbagai kolom spesifikasi. Dengan object, data yang kita kelola jadi punya "konteks" dan lebih manusiawi untuk dibaca.

Terakhir, semua data yang sudah kita kumpulkan dalam array maupun object tidak akan berguna jika kita tidak tahu cara mengolahnya secara otomatis. Di sinilah konsep Iterasi atau Perulangan masuk sebagai jembatan. Iterasi memungkinkan komputer untuk menyisir setiap laci di lemari arsip (array) atau membaca setiap kolom di kartu identitas (object) secara cepat dan akurat. Tanpa perlu memerintah satu per satu, kita bisa menyuruh program untuk melakukan kalkulasi atau pembersihan data pada seluruh kumpulan informasi tersebut hanya dengan beberapa baris kode perintah.

D.1 Konsep Utama

Konsep utama dari pertemuan 5 ini diantaranya:

1. Function, kalau di Teknik Industri itu ibarat SOP (Standard Operating Procedure). Kita cukup nulis instruksi perhitungannya sekali aja, lalu bisa dipanggil berkali-kali pas dibutuhkan tanpa harus menulis ulang logikanya dari awal
2. Array. Konsep ini digunakan untuk menyimpan sekumpulan data dalam satu wadah, contohnya seperti daftar kode produk yang cacat atau list stok material. Dapat dibayangkan seperti lemari arsip atau sheet di Excel
3. Object, digunakan untuk menyimpan sefat detail bsuatu barang. Object ini seperti Kartu Identitas (KTP) atau Form Spesifikasi Teknis sebuah mesin. Di dalemnya bisa menyimpan banyak informasi sekaligus, mulai dari ID mesin, merk, sampai statusnya lagi rusak atau jalan, jadi tiap item punya "identitas" yang jelas dan terstruktur

D.2 Keterkaitan dengan Teknik Industri

Materi dalam modul ini sangat relevan bagi mahasiswa Teknik Industri karena di era Industri 4.0, seorang insinyur tidak hanya dituntut untuk merancang sistem pabrik secara fisik, tetapi juga harus mampu mengotomasi dan memvisualisasikan data melalui integrasi sistem seperti IoT, ERP, dan MES. Pemrograman web dan mobile menjadi sarana vital untuk menyelesaikan masalah nyata di industri, mulai dari pencatatan produksi hingga pemantauan inventori secara digital. Dengan menguasai logika pemrograman, kita bisa membangun antarmuka yang mencegah *human error* serta meningkatkan produktivitas operator di lapangan.

Pada Pertemuan 5, konsep Function, Array, dan Object menjadi pondasi utama dalam mengelola data industri yang kompleks agar lebih terstruktur.
Berikut adalah dua skenario aplikasi nyata dari materi Pertemuan 5 dalam dunia Teknik Industri:

1. Otomasi Kalkulator Energi Listrik Mesin: Menggunakan konsep Function, kita dapat membuat alat hitung otomatis untuk konsumsi daya mesin (P = V x I). Skenario ini memungkinkan operator cukup memasukkan variabel tegangan dan arus, lalu sistem akan memberikan hasil daya mesin secara instan. Hal ini jauh lebih efisien dan akurat dibandingkan melakukan perhitungan manual yang berisiko salah input.
2. Sistem Monitoring Inventori Gudang Pintar: Dengan menggabungkan Array dan Object, kita bisa membuat sistem pengelolaan stok material. Misalnya, setiap material disimpan sebagai sebuah *Object* yang berisi kode, nama, dan jumlah stok, yang kemudian dikumpulkan dalam satu *Array* besar. Kita bisa menambahkan logika otomatis di mana sistem akan memberikan peringatan "Segera PO" jika stok material tertentu sudah di bawah batas minimum (safety stock), sehingga proses produksi tidak terhambat akibat kehabisan bahan baku.

Secara keseluruhan, penguasaan materi ini memungkinkan kita mentransformasi data mentah di lantai produksi menjadi informasi digital yang siap diolah untuk pengambilan keputusan manajerial yang lebih cepat

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

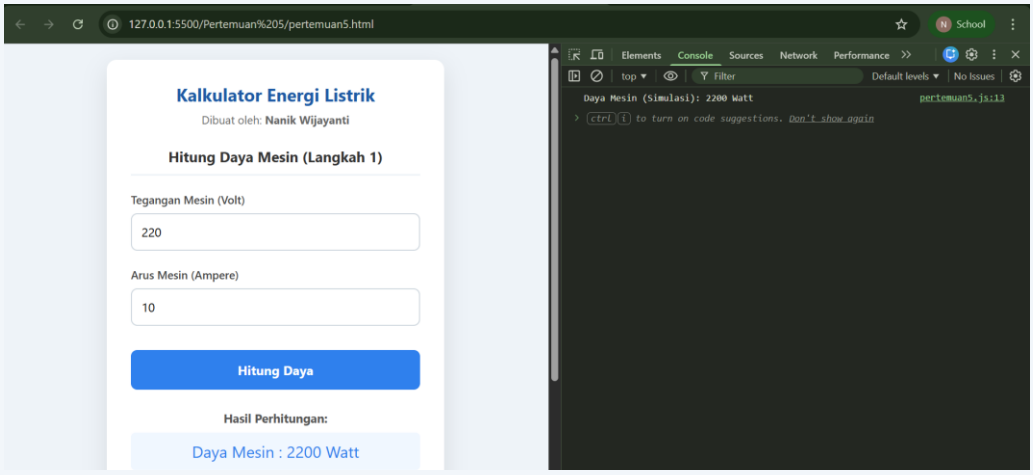
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat Function Perhitungan

Deskripsi singkat apa yang dilakukan pada langkah ini:buat file baru `pertemuan5.html` dan `pertemuan5.js`. Kita akan membuat kalkulator energi li

```
/* ======= KODE / OUTPUT LANGKAH 1 ======= */
// Langkah 1: Membuat Function Perhitungan
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}

// Pemanggilan Function untuk simulasi awal di Console
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin (Simulasi): " + totalDaya + " Watt");

// Fungsi ketika tombol diklik (Langkah 1 di HTML)
function prosesHitung() {
  let tegangan = Number(document.getElementById("teganganInput").value);
  let arus = Number(document.getElementById("arusInput").value);

  let daya = hitungDaya(tegangan, arus);

  document.getElementById("hasilDaya").innerText =
    "Daya Mesin : " + daya + " Watt";
}
```

Screenshot Hasil:



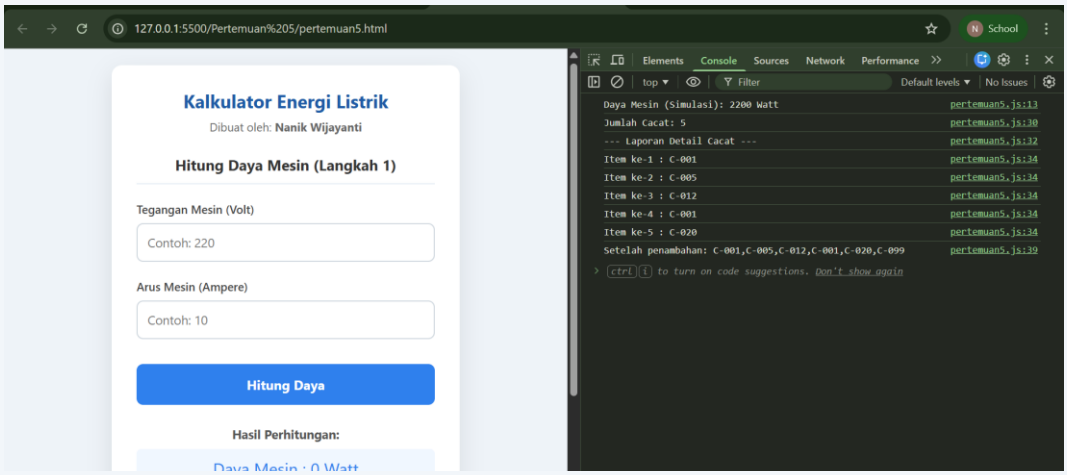
Gambar E.1 – Tampilan Hasil Membuat Function Perghitungan

E.2 Langkah 2 – Array dan Looping (Daftar Produk Cacat)

Menyimpan daftar kode produk yang cacat dalam satu shift.

```
/* ======= KODE / OUTPUT LANGKAH 2 ======= */
// Langkah 2: Array dan Looping (Daftar Produk Cacat)
// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001
terjadi 2 kali

console.log("Jumlah Cacat: " + daftarCacat.length);
// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
}

// Menambahkan data baru ke array
daftarCacat.push("C-099");
console.log("Setelah penambahan: " + daftarCacat);
```

Screenshot Hasil:



Gambar E.2 – Tampilan Hasil Data

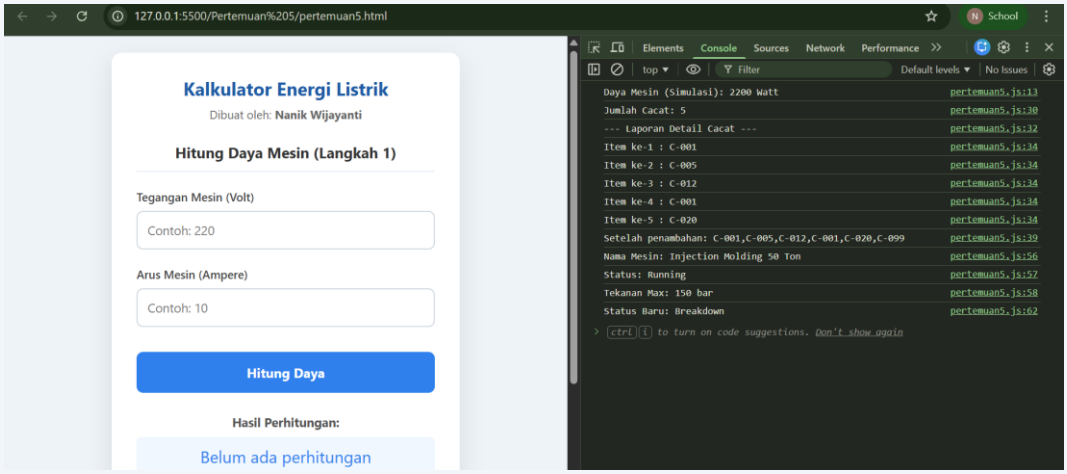
E.3 Langkah 3 – Object (Rekaman Data Mesin)

Representasi data mesin yang lebih kompleks.

```
/* ======= KODE / OUTPUT LANGKAH 3 ======= */

// Langkah 3: Object (Rekaman Data Mesin)
// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
```

```
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    tekananMax: 150, // bar
  },
};

// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");

// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);
```

Screenshot Hasil:



Gambar E.3 – Tampilan Hasil Rekaman Data Mesin

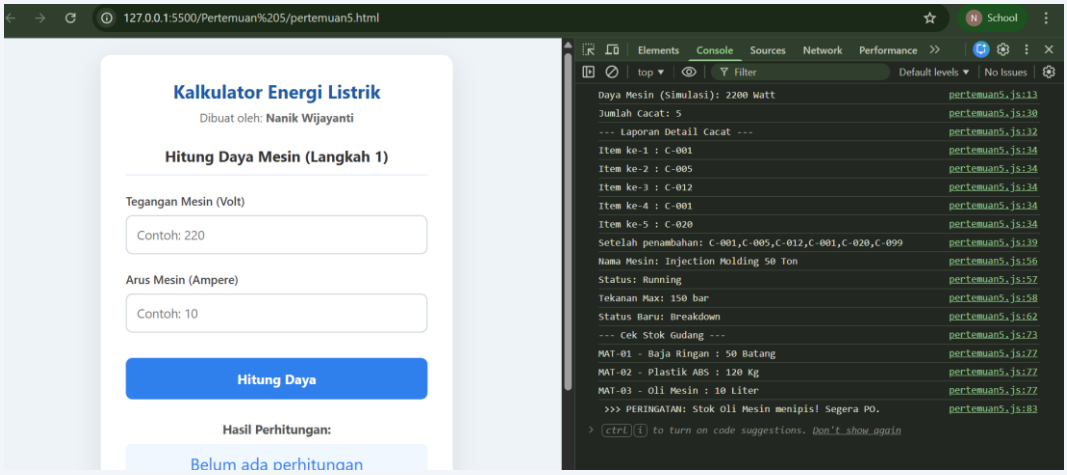
E.4 Langkah 4 – Gabungan (Array of Objects)

Ini adalah cara data biasanya disimpan di aplikasi nyata (mirip JSON).

```
/* ======= KODE / OUTPUT LANGKAH 3 ======= */

// Langkah 3: Object (Rekaman Data Mesin)
// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
```

```
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    tekananMax: 150, // bar
  },
};

// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");

// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);
```

Screenshot Hasil:



Gambar E4 – Tampilan Hasil Gabungan (Array of Objects)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Function

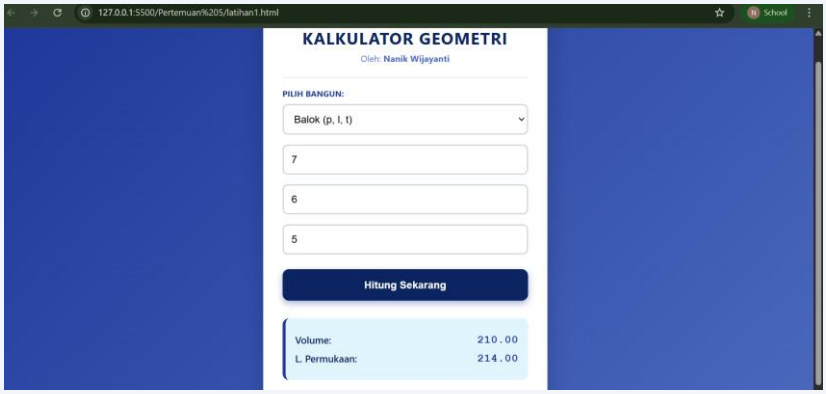
Pertanyaan / Instruksi Latihan:

> function `hitungLingkaran(jariJari)` yang mengembalikan nilai Luas dan Keliling. Tampilkan kedua nilai tersebut.

Jawaban / Kode Solusi:

```
// 1. FUNGSI UNTUK MENGATUR TAMPILAN INPUT (Muncul/Sembunyi)
function aturInput() {
  let pilihan = document.getElementById("pilihBangun").value;
  let v2 = document.getElementById("val2");
  let v3 = document.getElementById("val3");

  if (pilihan === "balok") {
    v2.style.display = "block";
    v3.style.display = "block";
  } else {
    v2.style.display = "none";
    v3.style.display = "none";
  }
}

// 2. LOGIKA PERHITUNGAN (Multiple Function)
function prosesHitungMulti() {
  let pilihan = document.getElementById("pilihBangun").value;
  let a = parseFloat(document.getElementById("val1").value);
  let b = parseFloat(document.getElementById("val2").value);
  let c = parseFloat(document.getElementById("val3").value);

  if (isNaN(a)) {
    alert("Isi angka dulu!");
    return;
  }

  let hasil1, hasil2, label1, label2;

  if (pilihan === "lingkaran") {
    hasil1 = (3.14 * a * a).toFixed(2);
    hasil2 = (2 * 3.14 * a).toFixed(2);
    label1 = "Luas: ";
    label2 = "Keliling: ";
  } else if (pilihan === "kubus") {
    hasil1 = (a * a * a).toFixed(2); // Volume s^3
    hasil2 = (6 * a * a).toFixed(2); // Luas Permukaan
    label1 = "Volume: ";
    label2 = "L. Permukaan: ";
  } else if (pilihan === "balok") {
    if (isNaN(b) || isNaN(c)) {
      alert("Isi lebar dan tinggi!");
      return;
    }
    hasil1 = (a * b * c).toFixed(2); // Volume p*l*t
```

```
    hasil2 = (2 * (a * b + a * c + b * c)).toFixed(2); // Luas Permukaan
    label1 = "Volume: ";
    label2 = "L. Permukaan: ";
  }

  // Update ke HTML
  document.getElementById("teksHasil1").innerHTML =
    label1 + "<span>" + hasil1 + "</span>";
  document.getElementById("teksHasil2").innerHTML =
    label2 + "<span>" + hasil2 + "</span>";

  // Laporan ke Console (Penting untuk dosen)
  console.log(`--- Perhitungan ${pilihan} ---`);
  console.log(`Nama: Nanik Wijayanti`);
  console.log(`${label1} ${hasil1}`);
  console.log(`${label2} ${hasil2}`);
}
```



Gambar F.1 – Tampilan Hasiil Kalkulator Geometri

F.2 Latihan 2 – Array Manipulation

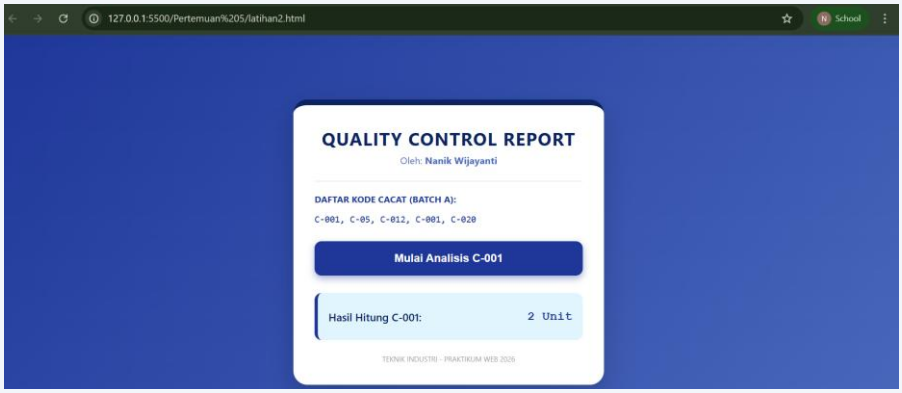
Pertanyaan / Instruksi Latihan:

> Dari array `daftarCacat` di atas, carilah berapa kali cacat "C-001" muncul (Gunakan loop dan if/counter).

Jawaban / Kode Solusi:

```
// 1. Inisialisasi Array sesuai perintah
let daftarCacat = ["C-001", "C-05", "C-012", "C-001", "C-020"];

// Menampilkan array ke halaman saat load
document.getElementById("tampil-array").innerText = daftarCacat.join(", ");
```

```
function hitungCacat() {
    // 2. Variabel Counter (Penghitung)
    let counter = 0;
    let target = "C-001";

    console.log("--- Laporan Detail Cacat ---");
    console.log("Jumlah Total Item: " + daftarCacat.length);

    // 3. Looping (Perulangan)
    for (let i = 0; i < daftarCacat.length; i++) {

        // Menampilkan proses di console
        console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);

        // 4. Kondisi IF untuk mengecek target
        if (daftarCacat[i] === target) {
            counter++; // Jika ketemu, counter bertambah
        }
    }

    // 5. Output ke tampilan HTML
    document.querySelector("#label-hasil span").innerText = counter + "
Unit";

    // Output ke Console
    console.log("Hasil akhir pencarian " + target + " : " + counter);
    alert("Analisis Selesai! Ditemukan " + counter + " cacat " + target);
}
```



Gambar F.2 – Tampilan Hasil Analisis Mesin

F.3 Tugas Proyek Mini – Sistem Antrian Job Shop

Deskripsi proyek mini:

Buat program simulasi antrian 3 pekerjaan mesin.

a. Buat Array of Objects bernama `antrianMesin`. Isi dengan 3 object job: `{idJob: "J01", namaProses: "Drilling", durasi: 30}`.

b. Buat function `prosesAntrian(antrian)` yang menerima parameter array tersebut.

c. Di dalam function, tampilkan pesan: "Memproses Job [ID] - [Nama] selama [durasi] menit" untuk setiap job dalam antrian.

d. Tambahkan 1 job baru ke array dan panggil function lagi.

Penjelasan pendekatan/solusi yang digunakan:

[Tuliskan penjelasan Anda tentang cara menyelesaikan proyek mini ini]

Dalam menyelesaikan proyek mini ini, fokus utama saya adalah menyatukan logika pemrograman dengan tampilan visual yang interaktif. Saya memulainya dengan merancang struktur data yang paling efisien untuk sebuah sistem antrian. Di sini, saya menggunakan Array of Objects untuk menyimpan daftar pekerjaan. Setiap tugas saya bungkus dalam sebuah Object supaya punya identitas lengkap, mulai dari ID, nama prosesnya, sampai durasinya. Dengan begini, data antar mesin tidak akan tertukar dan mudah kalau mau ditambah atau diubah.

Langkah selanjutnya, saya membuat sebuah "mesin utama" berupa ‘Function’ yang bertugas sebagai pusat pemrosesan. Saya membuat fungsi ‘prosesAntria’n agar kode saya tidak berantakan dan tidak perlu menulis perintah yang sama berkali-kali. Di dalam fungsi ini, saya menggunakan Iterasi (looping**)** untuk menyisir semua data di dalam Array tadi. Jadi, setiap kali ada data masuk, sistem otomatis membaca satu per satu dan menampilkan informasinya ke konsol browser.

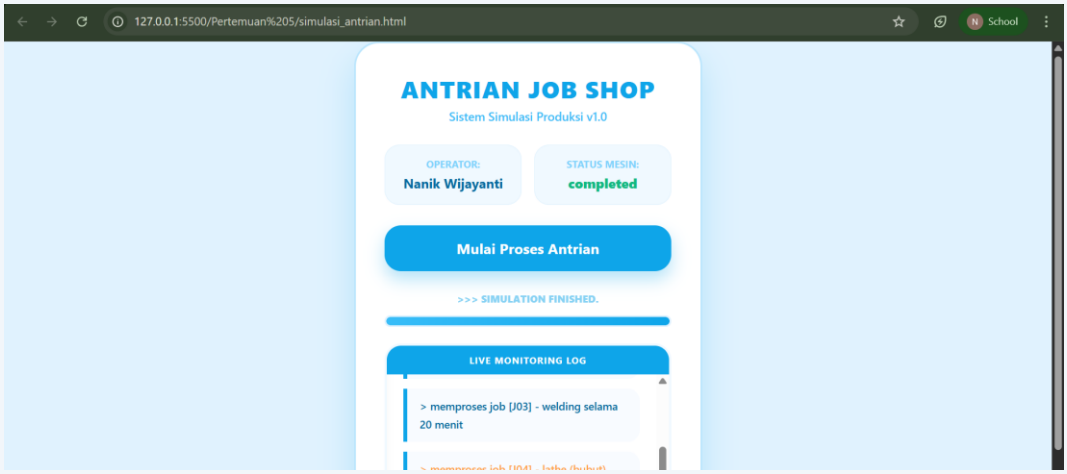
Saya juga menambahkan logika animasi menggunakan ‘setInterval’ untuk *loading bar*. Ini tujuannya supaya seolah-olah sistem sedang melakukan *scanning* database sebelum produksi dimulai. Saya juga memakai ‘setTimeout’ untuk mensimulasikan kejadian di lapangan, misalnya pas proses lagi jalan tiba-tiba ada "Job Baru" yang masuk ke antrian. Ini saya eksekusi pakai perintah .push() ke dalam Array, jadi daftar antriannya bisa bertambah secara dinamis.

Terakhir, saya menghubungkan semua logika tadi ke tampilan HTML melalui manipulasi DOM. Setiap ada perubahan status di JavaScript, entah itu *initializing*, *processing*, atau *completed,* teks dan warna di tampilan web akan ikut berubah secara otomatis.

Kode Utama:

```
// fungsi untuk mengolah data antrian dan menampilkannya
// bagian ini menangani looping dan pencetakan pesan ke console/layar
function prosesAntrian(antrian, isNew = false) {
    const logDiv = document.getElementById("logAntrian");

    // perulangan untuk mengambil setiap data job di dalam array
    antrian.forEach(function(job) {

        // susunan teks sesuai instruksi: memproses job [id] - [nama] selama
[durasi]
        let pesan = "> memproses job [" + job.idJob + "] - " + job.namaProses
+ " selama " + job.durasi + " menit";

        // cetak hasil ke console browser (sesuai poin c tugas)
        console.log(pesan);

        // bagian ini untuk memunculkan teks di monitor halaman web
        if(logDiv) {
            let itemLog = document.createElement("div");
            itemLog.className = "log-item";

            // kalau ini job tambahan, beri warna orange agar beda
            if(isNew) itemLog.style.color = "#ff9f43";

            itemLog.innerText = pesan;
            logDiv.appendChild(itemLog);
        }
    });
}

// fungsi utama yang mengatur jalannya simulasi dari awal sampai akhir
function jalankanSimulasiJob() {
    const logDiv = document.getElementById("logAntrian");
    const loader = document.getElementById("loader");
    const progress = document.getElementById("progress");
    const scanText = document.getElementById("scanText");
    const statusMesin = document.getElementById("statusMesin");

    // reset tampilan: kosongkan log dan munculkan animasi loading
    logDiv.innerHTML = "";
    loader.style.display = "block";
    scanText.style.display = "block";
    statusMesin.innerText = "initializing...";

    // logika untuk menggerakkan bar loading dari 0 ke 100%
    let lebarBar = 0;
```

```
    let timerLoading = setInterval(() => {
        if (lebarBar >= 100) {
            clearInterval(timerLoading);
            mulaiSimulasiData(); // lanjut ke proses data kalau loading
selesai
        } else {
            lebarBar += 2;
            progress.style.width = lebarBar + "%";
        }
    }, 30);

    // bagian inti yang berisi data job (array of objects)
    function mulaiSimulasiData() {
        statusMesin.innerText = "processing";
        scanText.innerText = ">>> database connected. starting
simulation...";

        // poin a: membuat daftar antrian awal (3 pekerjaan)
        let antrianMesin = [
            { idJob: "J01", namaProses: "drilling", durasi: 30 },
            { idJob: "J02", namaProses: "milling", durasi: 45 },
            { idJob: "J03", namaProses: "welding", durasi: 20 }
        ];

        // poin b & c: panggil fungsi untuk proses 3 data pertama
        prosesAntrian(antrianMesin);

        // poin d: simulasi tambah 1 job baru setelah jeda 1.5 detik
        setTimeout(() => {
            let jobBaru = { idJob: "J04", namaProses: "lathe (bubut)",
durasi: 60 };

            // masukkan ke array dan tampilkan hasilnya
            antrianMesin.push(jobBaru);
            prosesAntrian([jobBaru], true);

            statusMesin.innerText = "completed";
            scanText.innerText = ">>> simulation finished.";
        }, 1500);
    }
}
```



Gambar F.3 – Tampilan Hasil Simulasi Antrian

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Setelah saya jalankan simulasinya, hasil yang muncul di layar monitor maupun konsol sudah sangat sesuai dengan teori Function, Array, dan Object yang dipelajari di pertemuan ini.

Mengenai konsep Object dan Array. Di dalam kode, saya membuat variabel ‘antrianMesin’ yang berisi sekumpulan data pekerjaan. Hasilnya terbukti akurat; saat saya memanggil ‘job.idJob’ atau ‘job.namaProses’ dalam perulangan, program berhasil menampilkan detail spesifik seperti "J01", "drilling", hingga "welding". Ini membuktikan bahwa Object berhasil merepresentasikan entitas nyata secara detail, dan Array mampu menjadi wadah kolektif yang rapi untuk menyimpan objek-objek tersebut tanpa error saat live server.

Mengenai efisiensi Function (SOP). Saya menggunakan fungsi ’prosesAntrian’ untuk menangani logika pencetakan log. Hasilnya sesuai teori, saya tidak perlu menulis kode ’document.createElement’ berulang kali untuk setiap baris data. Cukup dengan satu fungsi, saya bisa memproses 3 data awal sekaligus, bahkan bisa memanggilnya kembali saat ada penambahan job baru (J04). Ini adalah penerapan nyata dari prinsip DRY (Don't Repeat Yourself) yang membuat kode saya jauh lebih ringkas.

Mengenai Iterasi dan Manipulasi Data. Saat simulasi berjalan, perulangan ’.forEach’ berhasil menyisir seluruh isi array dan mengeksekusi perintah secara berurutan. Bukti spesifiknya ada pada log yang muncul: program mencetak job secara urut dari J01 sampai J03. Selain itu, saat saya melakukan .push() pada ’jobBaru’, array ’antrianMesin’ terbukti bersifat dinamis karena bisa menerima data tambahan di tengah proses simulasi tanpa mengganggu data yang sudah ada sebelumnya.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kesalahan penulisan nama properti objek, di mana data tidak muncul atau hasilnya undefined saat dipanggil di fungsi.	Saya melakukan pengecekan ulang pada Object. Saya memastikan nama properti yang dibuat di array (misal: idJob) harus sama persis huruf besar-kecilnya saat dipanggil di dalam fungsi prosesAntrian.
File CSS atau JS tidak terbaca oleh browser (404 Not Found) karena adanya *typo* pada pemanggilan nama file di file HTML.	Saya menyamakan semua nama file dengan yang tertulis di tag <script> dan <link>. Saya memastikan tidak ada huruf yang hilang agar semua aset terhubung dengan sempurna.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan pemrograman yang saya pelajari di modul ini berkaitan erat dengan efisiensi di lantai produksi. Di Teknik Industri, bukan hanya berbicara mengenai mesin, tapi bagaimana mengatur aliran datanya supaya tidak terjadi *bottleneck*. Berikut adalah contoh skenario konkretnya di dunia Teknik Industri:

1. Sistem Monitoring Real-Time untuk Downtime Mesin
   Dalam skenario nyata, kita bisa pakai Object untuk mendata setiap mesin di pabrik (misalnya: ID mesin, status suhu, dan jam operasional). Kalau ada mesin yang tiba-tiba *overheat*, kita

bisa memakai Function untuk otomatis mengirim peringatan ke tim maintenance. Dengan Array, kita bisa merekap semua data kerusakan mesin dalam satu shift secara otomatis, sehingga tidak perlu lagi catat manual di kertas.

2. Optimasi Penjadwalan Produksi (Job Shop Scheduling)
   Seperti simulasi antrian pada proyek mini praktikum ke lima, teknik ini bisa diterapkan untuk mengatur antrian produk yang masuk ke berbagai stasiun kerja. Kita bisa membuat fungsi untuk menghitung mana pekerjaan yang harus didahulukan (misalnya memakai metode *Shortest Processing Time*). Dengan Iterasi, program akan otomatis menyusun jadwal dari ratusan orderan yang masuk ke sistem, sehingga *makespan* atau total waktu produksi menjadi jauh lebih singkat dan efisien.

3. Manajemen Inventaris dan Gudang Otomatis
   Teknik Array of Objects sangat berguna untuk mengelola stok barang di gudang. Setiap barang punya "kartu identitas" digital sendiri (Object). Kita bisa membuat fungsi yang otomatis mengurangi jumlah stok setiap ada barang keluar, dan kalau stoknya tinggal sedikit (di bawah reorder point), sistem akan langsung memberikan notifikasi. Hal ini jauh lebih akurat dibanding mengecek manual satu-satu ke rak gudang, yang pastinya lebih memakan waktu dan tenaga.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Saya belajar kalau Object itu berguna unruk membungkus detail mesin (seperti ID dan nama proses) jadi satu paket, jadi data setiap kerjaan tidak akan berantakan atau ketukar.
2	Penggunaan Array terbukti memudahkan dalam mengelola daftar antrian, karena kita bisa menambah atau menyimpan banyak data pekerjaan sekaligus dengan urutan yang jelas.
3	Melalui Function, saya bisa membuat sebuah prosedur kerja yang efisien. Saya hanya perlu menulis logika sekali, lalu bisa digunakan berkali-kali untuk memproses data yang berbeda.
4	Konsep Iterasi atau Perulangan sangat efektif untuk mengolah data secara otomatis; program dapat menyisir seluruh daftar antrian dan menampilkan laporannya tanpa perlu perintah manual satu per satu.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	[Tuliskan referensi ke-2 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]
[3]	[Tuliskan referensi ke-3 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]
[4]	[Tuliskan referensi ke-4 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Nanik Wijayanti NIM: 23051430006	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm | Bawah: 3 cm | Kiri: 4 cm | Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → … → Laporan Modul 12 → Laporan Modul 14 & 15.

c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```
PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)
```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	**Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS**

— Selamat mengerjakan —